DSA Assignment Document – Paavo Ahvenjärvi, Future Games (Umeå)

I chose binary search for the week 4 search algorithm. I found that the suspect is Alfred Pugsley.
I chose to make a hashmap for weeks 5 advanced data structure. How I made it is there is methods for the following: Calculatehash, Insert, GetValueInRange and GetValue. It should be pretty self explanatory what the following methods does, Insert for insertion of values, calculatehash to calculate the hash for the value, aka the hash function, GetValueInRange to get all values between two ranges inside of the hashmap which uses linear search, and GetValue which uses the same calculatehash to calculate where a value would be and searches there and it's sublist.
So if I send in a value it gets hashed in the CalculateHash and gets inserted into the array based on what it became, if I want to search for the value I just do it again but instead of putting in the value I tell the algorithm to search there instead for the value I'm searching for. Kinda like reverse engineering my way there.

I definitely had some challenges during this course, one of them would be how I implemented where the parsed json content was saved. In the beginning I had them saved in a tuple, but when I wanted to make something into EmployeeInfo it became a problem, because the tuple wasn't of type EmployeeInfo, so I had to rewrite some code so everything went from tuple to just an array of EmployeeInfo.
So basically making a semi dumb decision in the beginning and then having to rewrite some code and logic because of it nearer the end.

And then obviously making stuff generic. That was not fun to figure out. How I made week 4's search generic (binary search) was just using IComparable and using Functions with T and TKey.
I basically made it so you search for anything inside of the Binary search, and that meant you could put in the EmployeeInfo array aswell, but obviously you need to specify what you want to search on so that's where TKeys comes in play. I can basically tell the algorithm, search for this thing. If I don't want to search for a specific thing, send in only ints? That's fine aswell!

And well obviously IComparable is a way to compare the values in a generic way, you can't really just do == to compare values which are of type T/generic, so that's where I use that.

I did make my Quicksort algorithm generic aswell, because I thought it had to be generic for VG, so the binary search was easier to make generic because I had experience after the Quicksort one.
Implementing it for Quicksort was the hardest part, because it was the first time using TKeys etc. Why I used TKeys for Quicksort was because I want to specify what to sort on, if I sent the array with employees inside of the quicksort without specifying it would sort

them in alphabetical order, because it was using the default IComparable, which is not what we want.

I did make a hashmap in week 5 and why I chose the two hash functions I chose is because of the following:

Note that in the code there is like a function for everything, so basically 2 Calculate hash methods, two insert methods and so on, because obviously I needed to test both of the hash functions which needed all of the required methods for both sides.

The first hash function is one I just made, it converts the DateTime into the string format of years, months, days, hours, minutes, seconds ("yyyyMMddHHmmss"). After that it long parses it into a long (can't use int because it reaches integer overflow otherwise, because there is too many numbers for an int after the conversion) then I just modulo that with my hashMapLength which is 1009 (a prime number) and return it.

How my second hash function works is that it uses a very fancy formula I found online that is the following "hash * 33 + c". What it basically does is for every character (letter) in the input string it multiplies the current running hash value by 33 and adds the int value of the current char (letter) to it. So it becomes pretty unique. Then I just modulo that with hashMapLength which is again 1009 (a prime number) and return that.

Why I chose the first one is because it's the first method I came up with, pretty basic, turn into string into number. This is quite slow because you need to convert a DateTime into string and then into a long, which is a lot.

Why I chose the second one is because of how it's calculated. I feel like it calculates it way faster than converting into string and then into long and so on. Obviously it's still a lot of calculations you have to do because you are doing a foreach loop for every letter. But what I'm trying to say is that the second method is probably better if you want to ensure a more "unique" hash number and that is a bit faster.
Obviously the last number (after the modulo) is going to be in the range of 0-1008 but I think there's fewer collisions using the second hash function.
But for our data set it really depends, I think if you insert a datetime with like a bunch of milliseconds and everything into it, it will be slower because it has to go thru every single letter and make some crazy calculations, meanwhile the first one straight up converts it. But for a relatively "small" date, I would definitely choose the second one and I would say overall it's far better to have fewer collisions in the hashmap than it takes for it to calculate it's position in the hashmap once.

Also I'm also going to talk about what hash maps are good with. They are not good using any linear searches or "getting values in the range of" methods, but way better with just

one value searches which you know. So basically I wouldn't use a hashmap ever if I wanted to search for every value between x and y inside a hashmap. But if I knew what I wanted to search for, to check if x is in the hashmap, that's what I would use it for. Because then I can reverse engineer my way to that value because I know what I'm looking for. I can't reverse engineer my way thru values x and y without knowing if they are there.
If we take ints for example, it would be perfect for a hashmap if I wanted to know if there is a 5 inside of it. I can reverse engineer my way to it's place in the array by using the same hash function on 5. But if I want to know every number between 1-10, I can't do that, the computer doesn't know where anything is or if they are there, we are not specifying a specific place, so the only way to know that is to go thru every single element of the hashmap and compare them one by one to the condition, aka linear search, which is way slower.

Here are some performance benchmarks of everything in Ticks and milliseconds:

Quicksort: 3800 ticks, 0 milliseconds

Binary search: 35 ticks, 0 milliseconds

Hashmap insertion: 169 ticks, 0 milliseconds

Range gotten from the hashmap: 9795 ticks, 0 milliseconds

One value gotten from the hashmap: 3278 ticks, 0 milliseconds

Even with my probably very unoptimized hash function, the one value lookup is almost 3 times as fast as the range search.